

ADC-Aware Quantization of LLaMA-Family Models for In-Memory Computing

by

Alina Burykina

Master Thesis in Computer Science and Software Engineering

Submission: May 2, 2026

Supervisor: Prof. Dmitry Vetrov

Statutory Declaration

Family Name, Given/First Name	Burykina, Alina
Matriculation number	00000000
Kind of thesis submitted	Master Thesis

English: Declaration of Authorship

I hereby declare that the thesis submitted was created and written solely by myself without any external support. Any sources, direct or indirect, are marked as such. I am aware of the fact that the contents of the thesis in digital form may be revised with regard to usage of unauthorized aid as well as whether the whole or parts of it may be identified as plagiarism. I do agree my work to be entered into a database for it to be compared with existing sources, where it will remain in order to enable further comparisons with future theses. This does not grant any rights of reproduction and usage, however.

This document was neither presented to any other examination board nor has it been published.

German: Erklärung der Autorenschaft (Urheberschaft)

Ich erkläre hiermit, dass die vorliegende Arbeit ohne fremde Hilfe ausschließlich von mir erstellt und geschrieben worden ist. Jedwede verwendeten Quellen, direkter oder indirekter Art, sind als solche kenntlich gemacht worden. Mir ist die Tatsache bewusst, dass der Inhalt der Thesis in digitaler Form geprüft werden kann im Hinblick darauf, ob es sich ganz oder in Teilen um ein Plagiat handelt. Ich bin damit einverstanden, dass meine Arbeit in einer Datenbank eingegeben werden kann, um mit bereits bestehenden Quellen verglichen zu werden und dort auch verbleibt, um mit zukünftigen Arbeiten verglichen werden zu können. Dies berechtigt jedoch nicht zur Verwendung oder Vervielfältigung.

Diese Arbeit wurde noch keiner anderen Prüfungsbehörde vorgelegt noch wurde sie bisher veröffentlicht.

.....
Date, Signature

Abstract

This thesis investigates whether post-training quantization methods can be adapted to deploy LLaMA-family models in in-memory computing systems constrained by low-precision analog-to-digital converters (ADCs). Although prior work such as SmoothQuant and FlatQuant reduces activation and weight quantization error, ADC-constrained inference introduces an additional output quantization bottleneck at the matrix-vector multiplication stage. The work develops an ADC-aware evaluation pipeline for transformer linear layers, studies the limitations of standard post-training quantization in this setting, and explores modifications of FlatQuant that explicitly account for ADC behavior. The experimental part compares naive ADC quantization, baseline FlatQuant, and ADC-aware variants across different tile sizes, quantization precisions, and transform configurations. The main goal is to identify which parts of the LLaMA architecture are most sensitive to ADC constraints, whether post-training quantization alone is sufficient, and under which conditions lightweight adaptation may be required. The outcome of the thesis is intended to clarify the limits of reconstruction-only quantization methods for analog inference and to provide practical guidance for deploying large language models in IMC environments.

Contents

Abstract	iii
1 Introduction	1
2 Background and Related Work	2
2.1 In-Memory Computing and ADC-Constrained Inference	2
2.2 Quantization of Neural Networks	3
2.3 Challenges of Quantizing Transformer Models	4
2.4 Existing Methods	5
2.5 Research Gap	7
3 Problem Statement and Research Questions	8
3.1 Problem Definition	8
3.2 Research Questions	8
3.3 Scope and Assumptions	8
3.4 Evaluation Criteria	8
4 Proposed Method	9
4.1 Post-Training Quantisation	9
4.2 Quantisation-Aware Adaptation via Post-ADC Residual LoRA	12
5 Experiments and Results	14
5.1 Model, Data, and Hardware Setup	14
5.2 Evaluation Metrics	15
5.3 Bipolar ADC: Ablation Results	15
5.4 Unsigned (Unipolar) ADC: A Second Hardware Setting	21
6 Discussion	27
6.1 Interpretation of Results	27
6.2 Limits of PTQ Under ADC Constraints	27
6.3 Practical Implications	27
6.4 Threats to Validity	27
7 Conclusion	27
A Detailed Experimental Configurations	30
B Additional Figures and Tables	30
C Code Organization and Reproducibility Notes	30
D Key Code Listings	30
D.1 ADC Forward Pass	30
D.2 Kronecker Transform with Diagonal	31
D.3 Block-Level Loss with α -Mixing	31
D.4 Post-ADC Residual LoRA	32

1 Introduction

Large language models (LLMs) have become the dominant architecture for modern natural language processing (NLP), but their inference cost remains a major deployment obstacle. Transformer decoders are built primarily from large matrix–vector and matrix–matrix multiplications, and serving them requires repeatedly moving weights and activations between memory and compute units. In fact, autoregressive decoding is already memory-bandwidth-bound on modern GPUs: each forward pass reads billions of parameters to produce only a single output token, so the arithmetic intensity falls far below the hardware roofline and throughput is limited by memory bandwidth rather than compute [26]. This motivates hardware architectures that move computation closer to memory, eliminating the data movement bottleneck entirely, including in-memory computing (IMC) accelerators, where matrix–vector multiplication can be performed directly inside memory arrays rather than in a conventional digital compute core [34, 19, 6].

IMC is attractive for LLM inference because transformer layers are dominated by linear projections. In an idealized IMC pipeline, the weight matrix of a projection is stored in a memory array, an input activation vector is applied to the array, and the analog accumulation implements the dot product. However, this analog result must be converted back into a digital value before it can be consumed by the next layer, placing the analog-to-digital converter (ADC) directly on the neural network inference path. The precision and range of the ADC therefore determine how much information from the analog accumulation is preserved.

This thesis studies the interaction between transformer quantization and ADC-constrained analog inference. In a conventional digital quantization setting, the main sources of error are weight and activation quantization. In an IMC accelerator there is an additional output quantization stage: the accumulated analog result is passed through the ADC, which rounds it to the nearest multiple of a fixed step size Δ and clips it to the ADC output range. The central difference from digital low-bit inference is that even if weights and activations are quantized well, the accumulated output can still be severely distorted by a coarse ADC lattice.

Critically, the ADC step size is a hardware-defined constant determined by the tile size, the bit-widths of the activations and weights, the ADC resolution, and the ADC parallelism factor. It cannot be reduced by ordinary model calibration or by changing the learned quantization transform, making ADC-constrained inference a distinct optimization problem from standard digital PTQ.

LLMs are a particularly challenging target for this setting. Prior work has shown that transformer activations contain large outlier channels, and that these outliers are a major obstacle to low-bit quantization [11, 33, 30, 2, 23]. Because activation scales are computed from the per-token maximum, a single large channel sets the scale for the entire tile. Typical activation values then receive very small integer codes, reducing the effective density of the integer dot product. In an ADC pipeline this can produce poorly occupied ADC bins or dead regions where small accumulated outputs collapse to zero. LLM activation outliers therefore affect not only activation quantization, but also the quality of the analog output seen by the ADC.

This thesis focuses on LLaMA-family decoder-only transformer models, using Llama-3.2-1B as the main experimental target. The goal is to study whether transform-based post-training quantization (PTQ), combined with ADC-aware calibration and lightweight adap-

tation, can make low-bit LLaMA inference viable under ADC-constrained analog computation. The work starts from FlatQuant, a transform-based PTQ method that learns layer-wise affine transformations to make weights and activations easier to quantize [22]. The key idea is to change the coordinate system in which quantization is applied while preserving the linear map exactly, which is attractive for ADC-constrained inference because flatter activation distributions make per-token scaling less dominated by a few outlier channels.

Standard PTQ methods optimise for digital quantization and do not account for ADC output discretisation as a separate source of error. Even with well-calibrated weights and activations, ADC-path perplexity remains substantially higher than bypass perplexity — the quality achievable with digital quantization alone. Closing this gap requires calibration that is explicitly sensitive to ADC noise and, ultimately, a lightweight correction stage applied after the ADC path. This thesis makes the following contributions:

1. **ADC-constrained formulation for LLaMA PTQ.** The thesis formulates LLaMA inference under a tiled ADC-constrained MVM model and implements a simulation pipeline with per-token activation quantization, per-channel weight quantization, integer accumulation, ADC output quantization, and dequantization.
2. **Diagnosis of ADC-specific failure modes.** The thesis separates ordinary weight/activation quantization error from ADC output quantization error using bypass evaluation, dead-rate, ADC bin usage, reconstruction error, and layer/projection ablations. This shows that dead zones can be caused by poor transforms, but that the remaining degradation can be dominated by ADC output resolution.
3. **ADC-aware FlatQuant calibration.** The thesis extends FlatQuant with ADC simulation during calibration, propagated quantized inputs, partial propagation, bounded diagonal equivalent scaling, and staged training. These techniques substantially reduce ADC perplexity and make low-bit inference competitive with higher-bit baselines.
4. **Analysis of pure PTQ limitations.** Extensive ablations show that additional dead-zone penalties, aggressive PACT-style clipping, fixed Hadamard initialization, and bin-center losses are insufficient or harmful in the studied setting. Pure PTQ improves substantially but eventually reaches a plateau beyond which further calibration yields diminishing returns.
5. **Residual post-ADC low-rank correction.** The thesis introduces a lightweight residual correction stage on top of the frozen quantized ADC path, trained with distillation against a full-precision teacher. This stage closes the gap between low-bit ADC inference and higher-bit baselines while keeping the base ADC model frozen.

2 Background and Related Work

2.1 In-Memory Computing and ADC-Constrained Inference

In-memory computing (IMC) executes matrix–vector multiplication (MVM) directly inside memory arrays, reducing the cost of data movement that dominates conventional digital inference pipelines [34, 19, 6]. In crossbar-style accelerators, neural network weights are mapped to cell conductances, the input vector is applied as an analog excitation, and the array physics performs the accumulation. This is attractive for large neural networks

because MVM is the dominant operation in both attention and MLP blocks. However, the result of the array is analog, whereas the next layer expects a digital tensor. Therefore, each analog MVM must be followed by analog-to-digital conversion.

A simplified ADC-constrained inference pipeline can be written as

$$x_q = Q_x(x), \quad (1)$$

$$W_q = Q_w(W), \quad (2)$$

$$y_{\text{int}} = W_q x_q, \quad (3)$$

$$y_{\text{adc}} = Q_{\text{ADC}}(y_{\text{int}}), \quad (4)$$

$$\hat{y} = D_{\text{out}} y_{\text{adc}}, \quad (5)$$

where Q_x and Q_w are the activation and weight quantizers, Q_{ADC} denotes the output quantizer implemented by the ADC, and D_{out} is the dequantization map. In the ADC model used throughout this thesis, the analog accumulation is quantized by a coarse output lattice:

$$Q_{\text{ADC}}(u) = \Delta \cdot \text{clip}\left(\left\lfloor \frac{u}{\Delta} \right\rfloor, n_a, p_a\right), \quad (6)$$

where Δ is the ADC step size and (n_a, p_a) are the output code bounds. In a tiled IMC setting, one common hardware model yields

$$\Delta = \frac{2 n_t q_x q_w}{2^{b_a} k}, \quad (7)$$

where n_t is the tile size, b_a is the ADC bitwidth, k is the ADC parallelism factor, and q_x, q_w are the maximum activation and weight code levels. Equation (7) makes the hardware trade-off explicit: larger tiles or lower ADC precision lead to coarser output quantization.

This output quantization stage is qualitatively different from standard digital quantization. In purely digital low-bit inference, the dominant errors are usually input quantization, weight quantization, and approximate integer arithmetic. In IMC, even if Q_x and Q_w are well calibrated, the accumulated output can still be projected onto an insufficiently fine ADC lattice [5, 6]. Thus, the main bottleneck may shift from input-side quantization to output quantization. This distinction is central to the present thesis.

The problem becomes more severe for transformer models. Their hidden states are high-dimensional, anisotropic, and may contain rare channels with extremely large magnitude. These outliers stress the activation quantizer before the analog array and also distort the distribution of y_{int} presented to the ADC. Consequently, the question is not only how to quantize weights and activations, but also how to shape internal signals so that the analog accumulation uses the ADC range efficiently [5, 6].

2.2 Quantization of Neural Networks

Quantization maps continuous tensors to a finite set of discrete values in order to reduce memory, bandwidth, and arithmetic cost [18, 32, 16]. In symmetric uniform quantization, a scalar $x \in \mathbb{R}$ is mapped to

$$q(x) = \text{clip}\left(\left\lfloor \frac{x}{s} \right\rfloor, -q_{\text{max}}, q_{\text{max}}\right), \quad \hat{x} = s q(x), \quad (8)$$

where $s > 0$ is the quantization scale and $q_{\text{max}} = 2^{b-1} - 1$ for a signed b -bit quantizer. Symmetric quantization is widely used for weights and signed activations because it preserves zero exactly and simplifies integer arithmetic.

In asymmetric quantization, one adds a zero-point z :

$$q(x) = \text{clip}\left(\left\lfloor \frac{x}{s} + z \right\rfloor, q_{\min}, q_{\max}\right), \quad \hat{x} = s(q(x) - z). \quad (9)$$

This is useful when the signal distribution is non-negative or strongly shifted, as in some post-activation tensors or hardware-aware activation coding schemes [18, 32].

Quantization also differs in granularity. In per-tensor quantization, one scale is shared by the full tensor. In per-channel quantization, each output channel receives its own scale; this is particularly useful for weights because different rows of W often have very different ranges. For transformer activations, one also encounters per-token and per-group scaling, especially when the activation distribution changes strongly across tokens or heads.

Two training regimes dominate the literature. In post-training quantization (PTQ), a pre-trained floating-point model is quantized after training, usually using a small calibration set and a local reconstruction objective [15, 35, 20, 29, 12, 22, 1]. A generic PTQ objective for layer ℓ can be written as

$$\min_{\theta_\ell^q} \sum_{x \in \mathcal{C}} \left\| f_\ell(x; \theta_\ell) - \hat{f}_\ell(x; \theta_\ell^q) \right\|_2^2, \quad (10)$$

where $\mathcal{C}$ is a calibration set and θ_ℓ^q are the quantized parameters or auxiliary calibration variables. PTQ is attractive because it avoids retraining the full model.

In quantization-aware training (QAT), quantization is inserted into the forward pass during training or finetuning:

$$\min_{\theta} \sum_{(x,y)} \mathcal{L}(y, f_q(x; \theta)), \quad (11)$$

so that the parameters adapt directly to quantization noise [10, 14, 21, 8]. Since rounding is non-differentiable, QAT commonly relies on the straight-through estimator (STE), which uses the quantized function in the forward pass but approximates its gradient in the backward pass by

$$\frac{\partial q(x)}{\partial x} \approx 1. \quad (12)$$

Although heuristic, STE is one of the standard tools behind low-bit QAT and learned PTQ methods [10, 14].

Several refinements are especially relevant for the present work. PACT learns activation clipping thresholds [10]; LSQ learns the step size itself [14]; AdaRound learns non-trivial rounding decisions in PTQ [25]; and QDrop introduces stochastic activation dequantization during calibration to stabilize extremely low-bit PTQ [31]. These techniques all highlight the same theme: in low-bit settings, choosing the bitwidth is not enough; one must also decide how the signal is clipped, scaled, rounded, and propagated.

2.3 Challenges of Quantizing Transformer Models

Transformer language models are notoriously difficult to quantize. A major reason is the presence of activation outliers. LLM.int8() showed that hidden states in transformers contain rare but very large features, and that preserving these outlier dimensions can be critical even when most of the model is quantized aggressively [11]. The same general

phenomenon motivates SmoothQuant, Outlier Suppression+, QuaRot, SpinQuant, and FlatQuant [33, 30, 2, 23, 22].

This is especially problematic under max-based per-token scaling. For a signed activation tensor, a common choice is

$$s_x = \frac{\|x\|_\infty}{q_{\max}}, \quad (13)$$

which ensures that the largest feature maps to the largest available code. If one channel has magnitude M and the remaining channels are of order $\varepsilon \ll M$, then most quantization levels are allocated to that single channel. In low-bit settings, many of the smaller features are then quantized to zero or to a very narrow code band. In an ADC-constrained pipeline, this not only harms the input code density, but also leads to poorly occupied output bins after analog accumulation.

A second challenge is the sensitivity of the residual stream. Residual connections propagate local errors across many layers, so local reconstruction loss is not always predictive of end-to-end perplexity. QuaRot explicitly motivates its method by the need to remove outliers from the residual stream [2], and SpinQuant shows that choosing or learning a suitable rotation can dramatically change the accuracy of low-bit LLM inference [23]. QEP later argues that one of the central failures of standard layer-wise PTQ is the accumulation of quantization errors across layers, and proposes explicit quantization-error propagation to mitigate this effect [1]. CBQ reaches a related conclusion from a cross-block reconstruction perspective [12].

A third challenge is architectural non-uniformity. Not all transformer projections are equally sensitive. The output projections `o_proj` and `down_proj` inject their errors directly into the residual stream, whereas earlier projections such as `q_proj`, `k_proj`, `v_proj`, `up_proj`, and `gate_proj` are followed by additional mixing or nonlinear structure before their effect reaches the next block. This asymmetry is one reason why output-side projections often emerge as the most sensitive modules in low-bit transformer inference.

Finally, preserving hidden-state geometry is critical. A transform that reduces scalar range at the price of severely distorting the hidden-state subspace may still harm model quality. Rotation- and affine-transform methods are therefore attractive because they attempt to make the model more quantizable while preserving the underlying full-precision linear map. This point is especially relevant in the present thesis, where the goal is not merely to reduce local error, but to shape the signal path so that it is robust to both ordinary low-bit quantization and ADC quantization at the output of each analog MVM.

2.4 Existing Methods

Outlier-aware scaling and equivalent transformations. SmoothQuant reduces activation outliers by migrating part of the dynamic range from activations into weights through an equivalent transformation,

$$x' = xD^{-1}, \quad W' = WD, \quad (14)$$

where D is a diagonal scaling matrix chosen from activation and weight statistics [33]. This can dramatically improve W8A8 quantization for LLMs, and under ADC constraints it is appealing because it reduces input-side dominance before analog accumulation. Outlier Suppression+ extends this line with channel-wise shifting and scaling to better

handle activation asymmetry and concentration [30]. AWQ, although primarily a weight-only method, also exploits activation statistics to protect a small subset of salient weights [20]. OmniQuant generalizes equivalent-transform ideas by combining learnable weight clipping (LWC) and learnable equivalent transformation (LET), making PTQ more effective in very low-bit settings such as W4A4 [29]. The common limitation of this family is that it mainly addresses input-side quantization mismatch; it does not explicitly optimize the discrete ADC output lattice.

Reconstruction-based PTQ for LLMs. GPTQ formulates low-bit weight quantization as a second-order reconstruction problem and remains one of the strongest baselines for weight-only LLM quantization [15]. ZeroQuant combines PTQ with layer-wise distillation and hardware-oriented deployment strategies [35]. CBQ extends standard layer-wise PTQ to cross-block reconstruction [12], while QEP explicitly feeds quantized hidden states forward during PTQ in order to model accumulated error [1]. These methods are highly relevant to the present thesis because they move beyond purely local clean-input reconstruction. However, even when quantization error is propagated across layers, the main optimization target is still typically the reconstruction of digital hidden states rather than the occupancy and precision of the ADC output codes.

Rotation and geometry-based methods. QuaRot uses fixed rotations to eliminate hidden-state outliers without changing the full-precision transformer computation, enabling end-to-end 4-bit inference for weights, activations, and KV cache [2]. SpinQuant improves upon this by learning the rotations rather than drawing them at random, showing that the choice of rotation substantially affects low-bit LLM accuracy [23]. FlatQuant goes one step further and learns a layer-wise affine transform parameterized through Kronecker factors, allowing the model to flatten activation and weight distributions while keeping the transformed linear map equivalent to the original one [22]. These methods are highly relevant under ADC constraints because they preserve hidden-state geometry more carefully than pure clipping. Their limitation is that they were not originally formulated around the explicit analog output quantizer.

ADC-aware methods. RAOQ is directly motivated by the fact that output quantization in IMC introduces an additional bottleneck not captured by standard PTQ. It proposes activation shifting (A-shift), weight reshaping, and bit augmentation (BitAug) to improve the signal-to-quantization-noise ratio after ADC conversion [5]. Analog Foundation Models push this idea further and train LLMs to be robust to noisy, low-precision analog hardware with explicit input and output quantization constraints [6]. These methods are conceptually closest to the present thesis because they treat the ADC as part of the model rather than as a post hoc implementation detail. Their limitation is that they typically go beyond standard PTQ and require a stronger hardware-aware adaptation stage.

Lightweight post-quantization adaptation. When pure PTQ saturates, a natural next step is to keep the quantized backbone fixed and attach a small trainable correction branch. LLM-QAT shows that extremely low-bit transformer quantization often requires some form of QAT or finetuning to recover quality [21]. EfficientQAT improves the practicality of this idea with block-wise parameter updates and end-to-end training of quantization parameters [8]. In parallel, a line of low-rank methods—including INT2.1, LQER, Low-Rank Correction for Quantized LLMs, and LR-QAT—shows that much of the residual

quantization error can be captured by training only a small number of low-rank parameters [7, 4, 28, 3]. For ADC-constrained inference, this family is especially attractive: one can preserve a frozen quantized analog path and let a lightweight residual branch learn the remaining distortion. This is precisely the direction explored in the second stage of the present thesis.

2.5 Research Gap

The literature reviewed above establishes two complementary facts. First, state-of-the-art PTQ methods for LLMs—including scaling-based, rotation-based, and reconstruction-based approaches—are highly effective at reducing input-side activation and weight quantization error [33, 20, 29, 2, 23, 22]. Second, IMC-oriented work shows that analog output quantization introduces a separate and often dominant bottleneck that is not naturally captured by ordinary digital PTQ objectives [5, 6].

This creates a specific gap for ADC-constrained LLM inference. Standard PTQ methods typically optimize a local reconstruction objective

$$\mathcal{L}_{\text{PTQ}} = \sum_{x \in \mathcal{C}} \left\| f_{\ell}(x) - \hat{f}_{\ell}(x) \right\|_2^2, \quad (15)$$

possibly with clipping or affine preconditioning. Even when quantization-error propagation is included, the target is usually still a digital hidden-state reconstruction objective [1]. In an IMC accelerator, however, the effective layer map is

$$\hat{f}_{\ell}^{\text{ADC}}(x) = D_{\text{out}} Q_{\text{ADC}}(Q_w(W_{\ell}) Q_x(x)), \quad (16)$$

so the model is constrained not only by low-bit inputs and weights, but also by the ADC quantization of the analog accumulation. Consequently, a method can achieve strong ordinary PTQ metrics and still perform poorly once the ADC is inserted into the inference loop.

A second gap appears after pure PTQ saturates. Once the best transform-based ADC-aware PTQ solution has been found, the remaining error may no longer be well addressed by additional clipping, scaling, or rotation heuristics. At that point, the relevant question is whether the frozen ADC path can be corrected by a lightweight residual branch rather than by retraining the full model. Although low-rank correction has been studied in digital low-bit settings [4, 28, 3], its interpretation as a post-ADC residual correction on top of a frozen IMC signal path is not standard in the LLM PTQ literature.

The research gap of this thesis is therefore twofold:

1. existing PTQ methods mainly optimize input-side quantization error, while ADC-constrained inference introduces a distinct output quantization bottleneck that must be modeled explicitly;
2. once transform-based PTQ saturates, a lightweight residual correction stage may be required, but it is unclear where such a correction should be inserted and how it should be trained.

The following chapters address these gaps in two stages. First, they extend transform-based PTQ with ADC-aware objectives, error propagation, and bounded equivalent scaling. Second, they investigate low-rank residual correction on top of the frozen quantized ADC path, showing that post-ADC correction can recover a large fraction of the remaining accuracy loss in low-bit LLaMA inference.

3 Problem Statement and Research Questions

3.1 Problem Definition

Purpose. Formally define the problem studied in the thesis.

What to include. Introduce the linear operation in quantized code space, define the ADC-constrained output quantization step, and explain the roles of the main parameters such as activation precision, weight precision, ADC precision, tile size, and the hardware parameter k . Clearly state that the goal is to preserve model quality under these constraints.

3.2 Research Questions

Suggested research questions.

1. How strongly does limited ADC precision degrade the quality of LLaMA-family models compared with conventional quantization?
2. To what extent can a FlatQuant-style post-training approach mitigate this degradation?
3. Does reconstruction-only PTQ remain sufficient under ADC constraints, or are ADC-aware objectives required?
4. Which layers and projections are most sensitive to ADC quantization?
5. Under what conditions does the problem require lightweight model adaptation beyond PTQ?

3.3 Scope and Assumptions

Purpose. Clarify what the thesis covers and what it does not cover.

What to include. State which LLaMA-family models are evaluated, what hardware behavior is modeled, which parts of the inference pipeline are abstracted, and whether the study focuses on PTQ only or also includes limited adaptation baselines. This subsection is also the right place to state that the work relies on simulation rather than fabricated hardware.

3.4 Evaluation Criteria

Purpose. Define how success will be measured.

What to include. Include both model-level and ADC-level metrics. Model-level metrics may include perplexity or task accuracy. Internal metrics may include layerwise reconstruction error, clipping rate, dead-zone rate, bin utilization, or sensitivity by projection type.

4 Proposed Method

4.1 Post-Training Quantisation

4.1.1 Baseline: FlatQuant for LLaMA

FlatQuant [9] exploits the linear invariance of the matrix-vector product to redistribute the dynamic range of activations before quantisation. For any invertible transform $T \in \mathbb{R}^{d \times d}$,

$$y = xW^\top = \underbrace{(xT^{-1})}_{\tilde{x}} \underbrace{(TW^\top)}_{\tilde{W}^\top}.$$

Choosing T to flatten the distribution of $\tilde{x}$ improves both integer quantisation accuracy and ADC bin coverage. Computing a full $d \times d$ transform is prohibitive, so T is Kronecker-factorised as $T = T_1 \otimes T_2$ with $T_1, T_2 \in \mathbb{R}^{\sqrt{d} \times \sqrt{d}}$, reducing the parameter count from $\mathcal{O}(d^2)$ to $\mathcal{O}(d)$.

Calibration proceeds layer by layer in a teacher-student fashion. For each linear projection, a frozen floating-point (FP) teacher produces reference outputs $y^* = L_{\text{FP}}(x^*)$, and the quantised student $\hat{y} = \hat{L}_T(\hat{x})$ minimises the mean-squared error $\|\hat{y} - y^*\|^2$ over the Kronecker factors only, with weights held at their quantised values. After calibration, T is folded into the weights via reparameterisation; inference cost equals that of standard integer quantisation.

We adapt FlatQuant to the LLaMA-3 architecture [13] by wrapping all seven linear projections in each decoder block: q, k, v, o in the grouped-query attention sub-layer, and `gate`, `up`, `down` in the SwiGLU feed-forward network.

4.1.2 ADC Hardware Model and Forward Simulation

In an analog in-memory compute (AIMC) accelerator, quantised weights $\hat{w} \in [-q_w, q_w]^M$ are stored on a crossbar tile of size M . At inference, a quantised activation tile $\hat{x} \in [-q_x, q_x]^M$ is presented, the crossbar accumulates their integer dot product $y_{\text{int}} = \langle \hat{w}, \hat{x} \rangle$ in the analog domain, and an ADC samples the result with a fixed step size δ :

$$\hat{y} = \text{clip}\left(\left\lfloor \frac{y_{\text{int}}}{\delta} \right\rfloor, -2^{b_a-1}, 2^{b_a-1}-1\right).$$

The step size is a hardware constant set at manufacturing time:

$$\delta = \frac{2M q_x q_w}{2^{b_a} k},$$

where b_a is the ADC bit-width and k is the number of ADC channels operated in parallel. Two failure modes follow directly. The dead zone: if $|y_{\text{int}}| < \delta$, the floor operation maps the output to zero, discarding the signal entirely. Clipping: if $|y_{\text{int}}| > \delta \cdot (2^{b_a-1}-1)$, the output saturates. For our target configuration ($b_w = b_x = 4$, $M = 256$, $b_a = 8$, $k = 16$) this gives $\delta \approx 6.12$, coarse enough that a naïve INT4 baseline shows a bypass-to-ADC perplexity gap of $\times 153$ (15.41 vs. 2354).

4.1.3 Limitations of Per-Layer Reconstruction

Standard FlatQuant minimises per-projection MSE feeding each layer its clean FP activations from the calibration set. Two gaps make this insufficient under ADC inference.

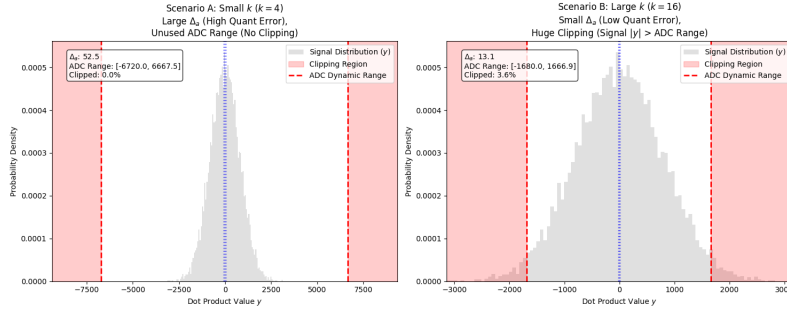


Figure 1: ADC dead-zone illustration. Inputs with $|y_{\text{int}}| < \delta$ are floored to zero, discarding the analog signal entirely. The two failure modes (dead zone and clipping) are determined by the fixed step δ and the b_a -bit ADC range.

First, the calibration objective is blind to the ADC step: a projection optimised in isolation may produce pre-ADC signals that fall inside the dead zone after re-quantisation of the next tile. Second, error accumulates across layers at inference but not during calibration — the input to layer ℓ at inference is the noisy, ADC-distorted output of layer $\ell-1$, not the clean FP tensor the optimiser saw. These gaps motivate all three ADC-aware extensions described below.

4.1.4 Block-wise Calibration with Propagation and α -Mixing

Block-level reconstruction. Rather than minimising per-projection MSE independently, we reconstruct the output of each entire transformer block:

$$\mathcal{L} = \|\hat{B}(\hat{x}) - B_{\text{FP}}(x^*)\|_2^2.$$

Gradients propagate through the full block ($q/k/v \rightarrow \text{attn} \rightarrow o \rightarrow \text{gate/up} \rightarrow \text{down}$), so the transforms of earlier projections are aware of how their error propagates to later ones within the same block.

Propagation across blocks. Blocks are calibrated sequentially from layer 0 to layer $N-1$. When training layer ℓ , its input is the ADC-quantised output produced by the already-trained layer $\ell-1$, not a clean FP activation. Error thus accumulates during calibration in the same way it does at inference, closing the distribution mismatch between training and deployment that per-layer calibration leaves open.

α -mixed objective. A purely ADC-propagated loss produces noisy gradients because the straight-through estimator must pass through multiple floor operations. We interpolate between the FP and ADC losses with a scalar $\alpha \in [0, 1]$:

$$\mathcal{L}(\alpha) = (1 - \alpha) \mathcal{L}_{\text{FP}} + \alpha \mathcal{L}_{\text{ADC}},$$

where $\mathcal{L}_{\text{FP}}$ uses the clean FP input and $\mathcal{L}_{\text{ADC}}$ uses the quantised input from the previous block. $\alpha = 0$ recovers clean calibration; $\alpha = 1$ is full propagation. An α -sweep (Section 5) shows that $\alpha = 0.5$ minimises ADC perplexity: below 0.5 the loss under-weights the ADC regime; above 0.5 gradient noise degrades transform quality. At $\alpha = 0.5$, INT8 ADC perplexity drops from 28.86 to 14.41 with no additional parameters.

4.1.5 Trainable Per-Channel Diagonals

We first establish a formal bound showing that orthogonal Kronecker transforms cannot suppress per-channel outliers beyond a fixed threshold, and that a diagonal factor is the precise missing degree of freedom.

Lemma 4.1 (Rotation-only outlier suppression bound). *Let $P = L \otimes R$ with $L^\top L = I$, $R^\top R = I$ be an orthogonal Kronecker transform on $\mathbb{R}^d$ ($d = a \cdot b$, $L \in \mathbb{R}^{a \times a}$, $R \in \mathbb{R}^{b \times b}$). For any $x \in \mathbb{R}^d$ define the effective participation ratio*

$$m_{\text{eff}}(x) = \frac{\|x\|_2^2}{\|x\|_\infty^2} \in [1, d].$$

Then for every orthogonal Kronecker transform P ,

$$\|xP\|_\infty \geq \frac{\|x\|_2}{\sqrt{d}},$$

and consequently the maximum achievable suppression of the peak activation over all such P satisfies

$$\frac{\|x\|_\infty}{\min_P \|xP\|_\infty} \leq \sqrt{\frac{d}{m_{\text{eff}}(x)}}.$$

Proof. Since P is orthogonal, $\|xP\|_2 = \|x\|_2$. For any vector $y \in \mathbb{R}^d$ the standard ℓ^∞ – ℓ^2 inequality gives $\|y\|_\infty \geq \|y\|_2/\sqrt{d}$. Applying this to $y = xP$ yields the first inequality. The suppression ratio bound then follows by dividing $\|x\|_\infty = \|x\|_2/\sqrt{m_{\text{eff}}(x)}$ by the lower bound $\|xP\|_\infty \geq \|x\|_2/\sqrt{d}$. $\square$

Corollary 4.2 (Kronecker capacity). *A full orthogonal matrix on $\mathbb{R}^d$ has $d(d-1)/2$ free parameters. The Kronecker family $L \otimes R$ has only $a(a-1)/2 + b(b-1)/2$ free parameters — for $d = 2048 = 32 \times 64$ this is 2512 vs. $\approx 2.1 \times 10^6$. Consequently, the Kronecker family is a strict low-dimensional subset of all orthogonal transforms and may not attain the bound in Lemma 4.1; the true achievable suppression ratio can be strictly smaller.*

Implications for INT4 ADC. In our setting, per-token activation scaling sets $s_x = \|x\|_\infty/q_{\text{max}}$. If a small number of channels persistently dominate (i.e. $m_{\text{eff}} \ll d$), Lemma 4.1 gives a ceiling on how much rotation alone can help. For example, with $d = 2048$ and 8 dominant channels ($m_{\text{eff}} \approx 8$), the maximum suppression factor is $\sqrt{2048/8} = 16$; if the required suppression exceeds this, rotation-only calibration is provably insufficient. The INT4 baseline confirms this empirically: the ADC perplexity gap of $\times 153$ persists even after FlatQuant rotation, whereas adding diagonal scaling reduces the dead-zone rate from 80.9% to 10.4%.

Motivation and formulation. A Kronecker rotation reshapes the covariance of an activation cloud but does not adjust per-channel scales. Under INT4, residual outlier channels remain after rotation: a few channels with magnitudes 5 – $10\times$ the median force the per-token scale factor to waste range on those channels and under-represent the rest, which directly worsens the ADC dead-zone rate because y_{int} per output channel depends on how uniformly codes are distributed.

We absorb a trainable diagonal $D = \text{diag}(d_1, \dots, d_c)$, $d_i \in [10^{-4}, 10]$, into the rotation:

$$\tilde{T} = T \cdot D.$$

The linear map is preserved by applying equal-and-opposite rescaling to the weights:

$$y = \underbrace{(x D^{-1} T^{-1})}_{\tilde{x}} \underbrace{(T D W^\top)}_{\tilde{W}^\top}.$$

The bounds on d_i prevent degenerate scaling. D is trained jointly with T during block-level calibration and folded into $\tilde{W}$ at deployment, adding no inference overhead. Crucially, because D is not orthogonal, it breaks the ℓ^2 -energy invariance assumed in Lemma 4.1: setting $d_j < 1$ directly reduces $\|x D\|_2$ in outlier channel j before the scale is computed, achieving suppression factors that no rotation can match.

Placement in attention and MLP blocks. We place separate diagonal matrices before the attention sub-layer (`diag_attn`, shared by $q/k/v$, plus a second instance before o) and before the FFN sub-layer (`diag_mlp`, shared by `gate/up`, plus a second before `down`). An ablation (Section 5) shows that both placements are necessary: `diag_mlp` drives the bypass-perplexity improvement while `diag_attn` closes the ADC gap; either alone leaves significant perplexity on the table.

4.1.6 Staged Training

Training all Kronecker factors and diagonals jointly from scratch with $\alpha = 0.5$ is unstable: attention gradients dominate early optimisation before MLP transforms have converged, yielding a poor initialisation for both sub-systems.

We split calibration into two stages. Stage A (30 epochs, learning rate η) trains only the MLP Kronecker factors $T_{\text{gate}}, T_{\text{up}}, T_{\text{down}}$ and MLP diagonals $D^{(\text{m})}$ with $\alpha = 0$ (clean FP inputs), providing a stable, noise-free landscape in which FFN transforms converge. Stage B (10 epochs, learning rate 0.1η) unfreezes the attention Kronecker factors T_q, T_k, T_v, T_o and attention diagonals $D^{(\text{a})}$, and switches to $\alpha = 0.5$. The MLP warm start prevents gradient interference from attention. This procedure achieves the best INT4 results among all pure-PTQ configurations: bypass perplexity 18.50, ADC perplexity 27.60.

4.2 Quantisation-Aware Adaptation via Post-ADC Residual LoRA

PTQ methods reach a floor around 27.5–28.5 ADC perplexity for INT4 regardless of calibration strategy. The limiting factor is ADC resolution: with $\delta \approx 6.12$ each output channel has at most ~ 30 usable discrete levels, introducing irreducible rounding error that no linear transform can eliminate. We complement PTQ with a lightweight adaptation step that learns to correct ADC distortion after it has occurred, in the digital domain, without modifying the frozen quantised model.

4.2.1 Architecture

The PTQ checkpoint is frozen entirely. For each of the seven linear projections in every decoder block, we add a parallel low-rank branch that reads the full-precision input x_{fp}

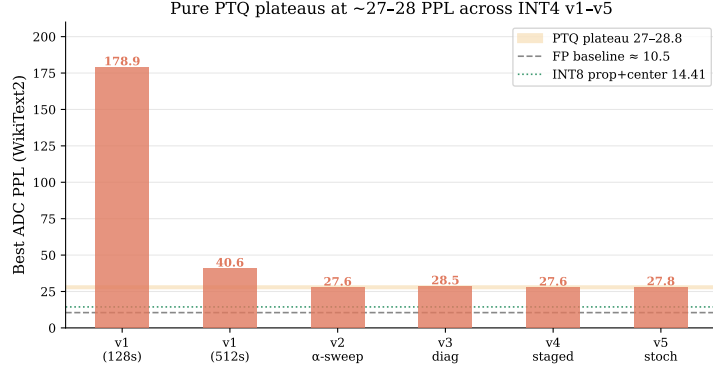


Figure 2: Pure PTQ plateau. After all transform-based PTQ improvements (sample count, α -mixing, diagonals, staged training), INT4 ADC perplexity saturates around 27.5–28.5, independent of calibration strategy. The remaining gap to the FP and INT8 baselines motivates the post-ADC residual correction stage.

and adds its output to the ADC result:

$$y = \underbrace{\text{ADC}_{\text{frozen}}(\hat{x})}_{\text{quantised path}} + \underbrace{\frac{\alpha_r}{r} B(A(x_{\text{fp}}))}_{\text{residual correction, rank } r},$$

where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ are LoRA [17] factors stored and computed in `float32`, and α_r/r is the standard LoRA scaling. All base model parameters — weights, Kronecker factors, diagonals, and ADC step sizes — remain frozen throughout. For Llama-3.2-1B ($d = 2048$, 16 decoder blocks, 7 projections each), rank-4 adapters add $\sim 1.8\text{M}$ parameters (0.15% of the total model).

4.2.2 Training Objective

The adapters are trained for 5 epochs with learning rate 10^{-4} on the same 128-sample calibration set used for PTQ. The loss combines cross-entropy on the model’s predictions with a knowledge-distillation term towards the FP teacher:

$$\mathcal{L}_{\text{LoRA}} = \text{CE}(\hat{p}, y) + \lambda \text{KL}(\hat{p} \parallel p_{\text{FP}}),$$

with $\lambda = 1$. The FP teacher forward pass is computed once per batch with no gradient. Ablations (Section 5) show that the KL term improves ADC perplexity by ~ 0.6 points over CE alone, and that rank $r = 4$ saturates the gain — rank 8 yields only marginal further improvement. Restricting adapters to the last eight layers is significantly worse than covering all sixteen, confirming that ADC distortion accumulates across the full depth of the network.

4.2.3 Why Post-ADC Placement

Placing the LoRA correction before the ADC — modifying the pre-ADC integer signal — fails catastrophically. Any correction $\Delta = BAx$ is itself quantised by the fixed step δ before reaching the output, so it cannot express sub- δ adjustments. Moreover, gradients

through the floor operator are too noisy for stable training from a cold start, and the coupled interaction with the hard ADC clamp causes divergence.

Post-ADC placement avoids both problems: the correction is added in the digital domain after the ADC read, at full floating-point precision, with clean gradients from the CE+KL loss. ADC perplexity drops from the PTQ plateau of 27.60 to 13.96 for INT4 on WikiText-2, surpassing the INT8 PTQ result (14.41), and generalises to C4 (ADC perplexity 23.30 vs. PTQ baseline 46.40).

5 Experiments and Results

5.1 Model, Data, and Hardware Setup

Model. All experiments use Llama-3.2-1B [13] in `bfloat16`. The model has 16 decoder layers, hidden dimension $d = 2048$, 32 attention heads with grouped-query attention, and a SwiGLU FFN with intermediate dimension 8192. It is small enough to run a full sweep of dozens of configurations overnight on a single GPU while being representative of the transformer architecture used in larger LLaMA-family models.

Calibration data. FlatQuant transforms and LoRA adapters are calibrated on WikiText-2 training text [24]. Unless stated otherwise, 128 randomly sampled sequences of 2048 tokens each are used. Selected experiments use 512 or 1024 samples to study the effect of calibration set size; the INT4 series demonstrates that sample count is a dominant lever for this bit-width.

Evaluation. Perplexity is measured on the WikiText-2 test set using a sliding window of context length 2048 and stride 1024. Selected configurations are additionally evaluated on C4 [27] under the same protocol to verify out-of-domain generalisation.

ADC hardware parameters. Table 1 lists the fixed hardware constants used throughout. All values are chosen to match a plausible near-term AIMC chip target. The ADC step size δ is fully determined by these constants via $\delta = 2Mq_xq_w/(2^{b_a}k)$, giving $\delta \approx 2016$ for INT8 and $\delta \approx 6.12$ for INT4.

Table 1: Fixed ADC hardware parameters.

Parameter	Symbol	Value
Tile size (crossbar columns)	M	256
ADC bit-width	b_a	8
Parallel ADC channels	k	16
INT8 weight range	q_w	127
INT8 activation range	q_x	127
INT4 weight range	q_w	7
INT4 activation range	q_x	7
ADC step (INT8)	δ	≈ 2016
ADC step (INT4)	δ	≈ 6.12

FlatQuant calibration settings. Transforms are trained for 30 epochs with learning rate 5×10^{-3} using the AdamW optimiser. Stage B of staged training uses 10 epochs at 5×10^{-4} . Static activation scales for Step 2 of the FlatQuant pipeline use the 99.9th-percentile clipping strategy (`calibration.method=percentile`).

5.2 Evaluation Metrics

All experiments are characterised by the following metrics.

- **PPL (bypass).** WikiText-2 perplexity with ADC disabled; only tiling and integer quantisation are active. Measures FlatQuant transform quality in isolation. Lower is better; the FP baseline is ≈ 10.5 (INT8 regime) and $\approx 10.5 / \approx 14$ depending on bit-width.
- **PPL (ADC).** WikiText-2 perplexity with the full ADC pipeline enabled. This is the primary metric; the bypass-ADC gap measures the additional degradation introduced by finite-resolution ADC.
- **dead_rate.** Fraction of output channels across all layers and calibration tokens for which $|y_{\text{int}}| < \delta$, i.e. the ADC output is identically zero. Measured on the calibration set; lower is better.
- **clip_rate.** Fraction of ADC outputs that saturate at the maximum code $\pm(2^{b_a-1} - 1)$. High clip rate indicates the integer accumulation systematically overflows the ADC range.
- **bin_usage.** Fraction of the 2^{b_a} ADC output bins that are actually occupied, averaged over layers and tokens. Higher is better; near-zero bin usage indicates the ADC is effectively acting as a 1-bit comparator.
- **reconstruction_rel.** Relative mean-squared error between the ADC output and the FP reference, $\|\hat{y} - y^*\|^2 / \|y^*\|^2$, on the calibration set. Tracks layer-level signal fidelity independently of perplexity.

5.3 Bipolar ADC: Ablation Results

All results below use the bipolar ADC with signed accumulation $y_{\text{int}} \in [-M, +M]$ and the hardware constants from Table 1. FP reference: WikiText-2 PPL 10.5. All tables report WikiText-2 perplexity unless stated otherwise.

5.3.1 INT8: Effect of Calibration Objective

Table 2 compares four INT8 configurations at 128 calibration samples, isolating the contribution of block-wise propagation and the bin-center auxiliary loss.

Propagation alone reduces ADC PPL by 47% ($28.86 \rightarrow 14.55$) with no change to `dead_rate`, confirming that the remaining gap is ADC resolution, not the dead zone. The bin-center loss has zero effect on its own but gives a marginal improvement (+0.14 PPL) on top of propagation.

Table 2: INT8 calibration study. Llama-3.2-1B, 128 samples, $\delta \approx 2016$.

Configuration	Bypass PPL	ADC PPL	dead_rate
FP baseline	—	10.5	—
Baseline ($\alpha = 0$)	10.01	28.86	10.3%
Bin-center ($\lambda = 0.1$)	10.01	28.86	10.3%
Propagated ($\alpha = 1$)	11.01	14.55	10.3%
Prop + bin-center	11.00	14.41	10.3%

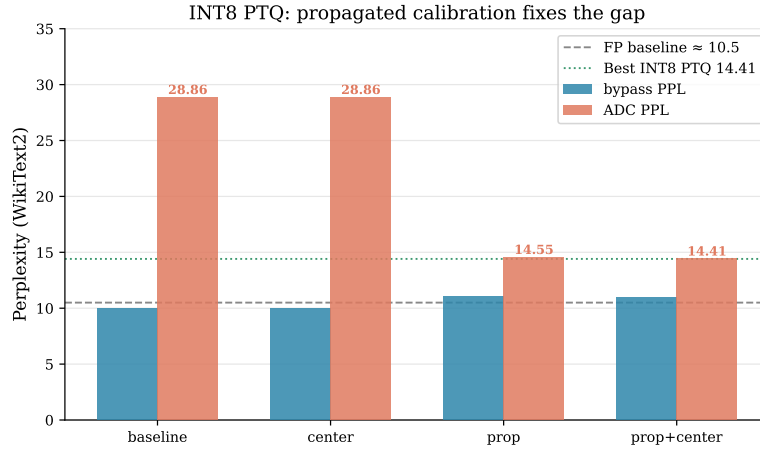


Figure 3: INT8 calibration ablation. Bin-centre loss alone has no effect; propagated calibration drops ADC PPL by 47% ($28.86 \rightarrow 14.55$) by exposing the calibration to the same ADC-distorted inputs that the model sees at inference.

5.3.2 INT4: Sample Count and Propagation

The INT4 baseline exhibits a bypass–ADC gap of $\times 153$ (15.41 vs. 2354.86), far wider than INT8. Table 3 shows that 128 calibration samples are insufficient for propagation, while 512 samples combined with propagation collapse the gap to $\times 1.2$.

Table 3: INT4 sample count and propagation sweep. $\delta \approx 6.12$.

Config	Samples	Bypass PPL	ADC PPL	Gap
Baseline	128	15.41	2354.86	$\times 153$
Propagated ($\alpha = 1$)	128	40.28	203.89	$\times 5.1$
Hadamard init + prop	128	34.73	178.86	$\times 5.1$
Decoupled (train w8, eval w4)	128	11.64	276.25	$\times 24$
Propagated ($\alpha = 1$)	512	34.25	40.63	$\times 1.2$
Baseline	1024	19.80	56.83	$\times 2.9$

Hadamard initialisation provides no benefit over random initialisation at the same sample count. Decoupled training (transforms optimised for INT8, evaluated at INT4 ADC) achieves the best bypass PPL (11.64) but a poor ADC gap ($\times 24$) because the transforms are not calibrated to the INT4 δ .

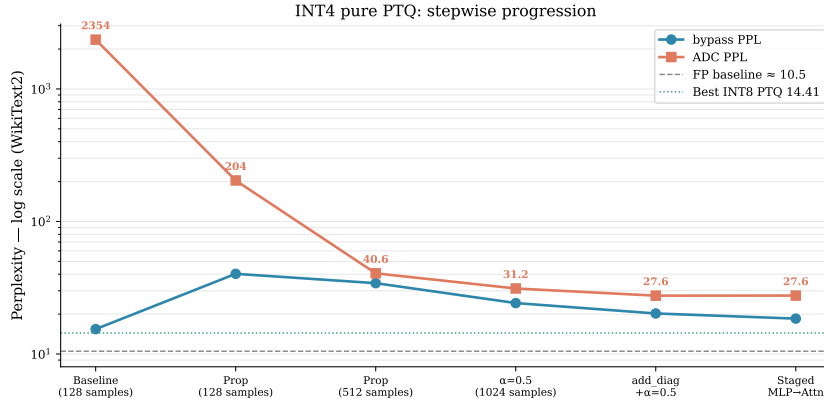


Figure 4: Step-by-step INT4 PTQ progression. Each step targets one failure mode: 128 samples (catastrophic, ADC PPL 2355); propagation + 1024 samples (40.6); partial $\alpha = 0.5$ mixing and bounded diagonals (27.56); staged training (27.60 with cleanest bypass 18.50). FP baseline and INT8-prop+centre reference lines are shown.

5.3.3 INT4: α -Mixing Sweep (1024 Samples)

Table 4 sweeps α from 0 to 1 at 1024 samples and evaluates two additional configurations: two-stage training and the diagonal extension.

$\alpha = 0.5$ gives the best gap ratio ($\times 1.29$); below 0.5 the ADC regime is under-weighted, above 0.5 gradient noise from the floor operator degrades transform quality. Adding the diagonal (bounded LET, $d_i \in [10^{-4}, 10]$) with $\alpha = 0.5$ achieves the best absolute ADC PPL (27.56) across all pure-PTQ configurations.

Table 4: INT4 α -mixing sweep. 1024 calibration samples.

Config	α	Bypass PPL	ADC PPL	Gap
Baseline	0.00	19.80	56.83	$\times 2.87$
Prop $\alpha = 0.25$	0.25	21.65	32.35	$\times 1.49$
Prop $\alpha = 0.5$	0.50	24.24	31.22	$\times 1.29$
Prop $\alpha = 0.75$	0.75	27.46	35.92	$\times 1.31$
Prop $\alpha = 1.0$ (full)	1.00	30.32	40.20	$\times 1.32$
2-stage (A: no prop, B: $\alpha = 0.5$)	—	22.82	32.21	$\times 1.41$
Diag + $\alpha = 0.5$	0.50	20.23	27.56	$\times 1.36$

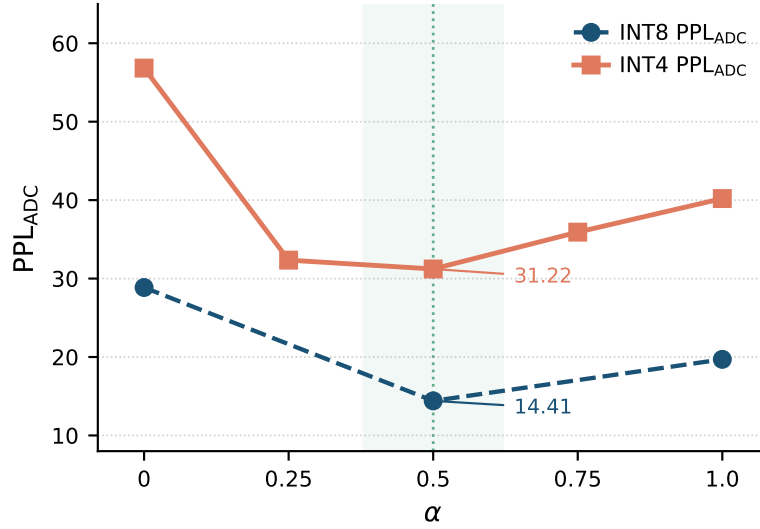


Figure 5: α -mixing sweep at 1024 calibration samples. ADC PPL is minimised at $\alpha = 0.5$; smaller values under-weight the ADC regime, larger values amplify gradient noise from the floor operator and degrade bypass PPL.

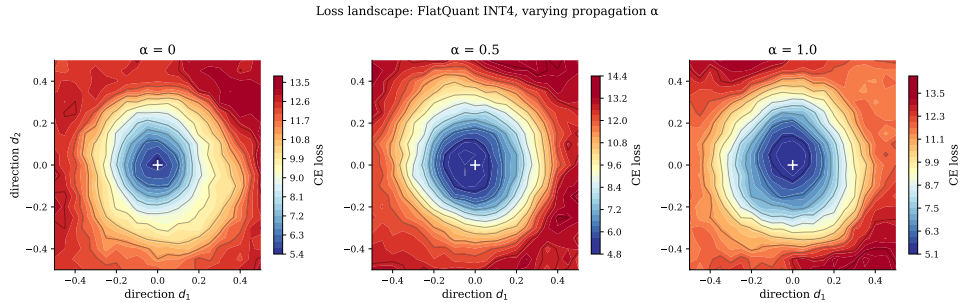


Figure 6: Loss-landscape visualisation around the converged transforms for $\alpha \in \{0, 0.5, 1\}$ via filter-normalised random perturbations. The $\alpha = 0.5$ minimum is the widest and flattest, indicating that mixed FP/ADC calibration finds smoother solutions than either extreme.

5.3.4 INT4: Diagonal Placement, MLP Decomposition, and Staged Training

Table 5 ablates diagonal placement (v3) and decomposes the MLP diagonal into its two components (v4), identifying the `down_proj` as the key driver.

Table 5: INT4 diagonal placement and staged training. 1024 samples, $\alpha = 0.5$.

Config	Bypass PPL	ADC PPL	Note
Both diag (control)	19.69	28.46	reproduced v2
Attn diag only	24.98	29.41	worse bypass
MLP diag only	19.37	31.65	best bypass, wide gap
MLP diag + layer-wise α	19.00	28.66	best bypass overall
Both diag + layer-wise α	19.39	28.55	no gain over flat α
up_gate diag only	23.22	39.59	weak
down diag only	20.42	31.26	key driver
Staged: MLP diag $\rightarrow$ attn diag	18.50	27.60	best PTQ

The `down_proj` diagonal drives nearly all of the MLP bypass improvement; `up_gate` contributes little. Staged training (Stage A: MLP diag, no prop; Stage B: attn diag, $\alpha = 0.5$) achieves the best combined result — bypass 18.50 and ADC 27.60 — by avoiding gradient interference between the two sub-systems.

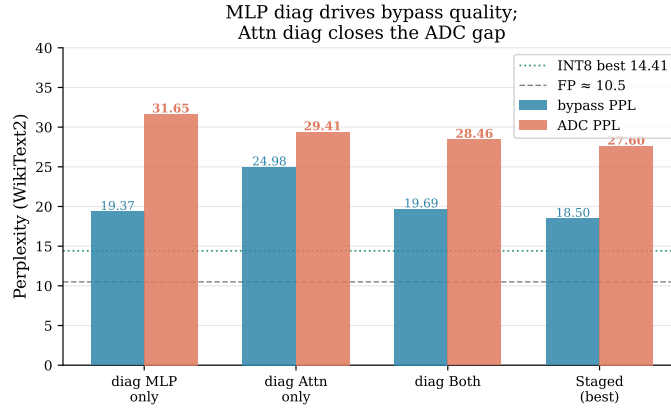


Figure 7: Roles of the MLP and attention diagonals. The MLP diagonal drives bypass-PPL improvements (transform quality); the attention diagonal closes the bypass-ADC gap (robustness to ADC noise). Either alone leaves significant perplexity on the table; staged training combines both.

5.3.5 INT4: Stochastic Propagation

Table 6 compares deterministic $\alpha = 0.5$ against two stochastic mixing schedules, in flat and staged settings.

Bernoulli switching yields the best bypass (16.94) but the worst ADC gap — batches that use only the clean FP path provide no ADC signal to the attention transforms. Beta(2,2)

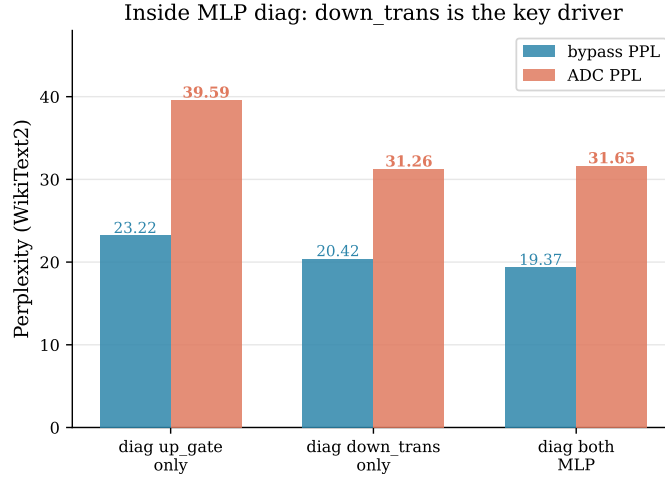


Figure 8: MLP-diagonal decomposition. `down_proj` contributes nearly all of the bypass improvement; the `up_gate` diagonal contributes little, since `down_proj` sits immediately before the next ADC accumulation and shapes the signal that is read.

Table 6: INT4 stochastic propagation. 1024 samples.

Schedule	Mode	Bypass PPL	ADC PPL	Gap
Deterministic $\alpha = 0.5$	flat	24.24	31.22	$\times 1.29$
Bernoulli (p=0.5)	flat	19.25	35.38	$\times 1.84$
Beta(2,2)	flat	22.90	31.40	$\times 1.37$
Deterministic $\alpha = 0.5$	staged	18.50	27.60	$\times 1.49$
Bernoulli (p=0.5)	staged	16.94	32.44	$\times 1.92$
Beta(2,2)	staged	17.16	27.82	$\times 1.62$

samples near 0.5 each batch and closely matches deterministic $\alpha = 0.5$; the marginal ADC gain (27.82 vs. 27.60) does not justify the added complexity.

5.3.6 Post-ADC Residual LoRA: Initial Sweep

Table 7 evaluates LoRA adapters on top of the best PTQ checkpoint (staged, ADC PPL 27.60), varying rank and target projection sets.

Table 7: LoRA initial sweep (v6). Base PTQ: bypass 18.50, ADC 27.60.

Config	Targets	Bypass PPL	ADC PPL	Gap
No LoRA (base PTQ)	—	18.50	27.60	$\times 1.49$
LoRA $r = 4$, down	1 proj	19.33	16.32	$\times 0.84$
LoRA $r = 8$, down	1 proj	19.17	15.79	$\times 0.82$
LoRA $r = 4$, down+o	2 proj	18.68	15.63	$\times 0.84$
LoRA $r = 4$, all 7 proj	7 proj	17.73	16.53	$\times 0.93$

Across all LoRA configurations ADC PPL falls below bypass PPL (gap < 1), confirming that the adapter learns to specifically correct ADC noise. Adding o_proj to down_proj gives the best ADC PPL (15.63); covering all 7 projections gives the best bypass (17.73) at the cost of a slightly wider gap.

5.3.7 Post-ADC Residual LoRA: Ablation Study

Table 8 reports the full ablation across rank, loss, layer coverage, and adapter placement (v7, all runs use $r = 4$, down+o unless stated).

Rank saturates at $r = 4$ (only 0.33 PPL gap to $r = 8$). CE+KL outperforms CE alone by 0.64 PPL with down+o targets and by 2.65 PPL with all-7 targets — the FP teacher signal is most valuable when the adapter covers more projections. Earlier layers carry more accumulated ADC error: first-8 (16.64) beats last-8 (20.08). Pre-ADC placement diverges catastrophically (ADC PPL 105.6) because the correction is itself quantised by the fixed δ . The best single configuration is **all-7, CE+KL**, $r = 4$: ADC PPL 14.03, beating the INT8 PTQ result (14.41).

5.3.8 Post-ADC Residual LoRA: Generalisation to C4

Table 9 evaluates the top-3 configurations on both WikiText-2 and C4 to test out-of-domain transfer.

CE+KL configurations reduce C4 ADC PPL by 50% relative to the PTQ baseline (46.40 $\rightarrow$ 23.30) despite being calibrated solely on WikiText-2. CE-only ($r = 8$, down+o) shows much weaker C4 generalisation (29.39 vs. 23.30), confirming that the KL teacher signal is the key ingredient for cross-domain transfer. All-7 CE+KL is the best overall checkpoint on both datasets.

5.4 Unsigned (Unipolar) ADC: A Second Hardware Setting

All experiments described so far assume a bipolar ADC: the analog crossbar accumulates a signed integer dot product $y_{\text{int}} \in [-M, +M]$ and the ADC reads both positive and

Table 8: LoRA ablation study (v7). Llama-3.2-1B, INT4.

Axis	Setting	ADC PPL
Seed	seed 1	15.79
	seed 2	15.88
	seed 3	15.93
Rank	$r = 1$	15.90
	$r = 2$	15.60
	$r = 4$	15.57
	$r = 8$	15.24
Loss	CE only	14.60
	CE + KL	13.96
Layer coverage	first 8 layers	16.64
	all 16 layers	15.63
	last 8 layers	20.08
Placement	post-ADC (proposed)	15.63
	pre-ADC	105.63 (<i>diverged</i>)
All 7 proj	CE only	16.68
	CE + KL	14.03

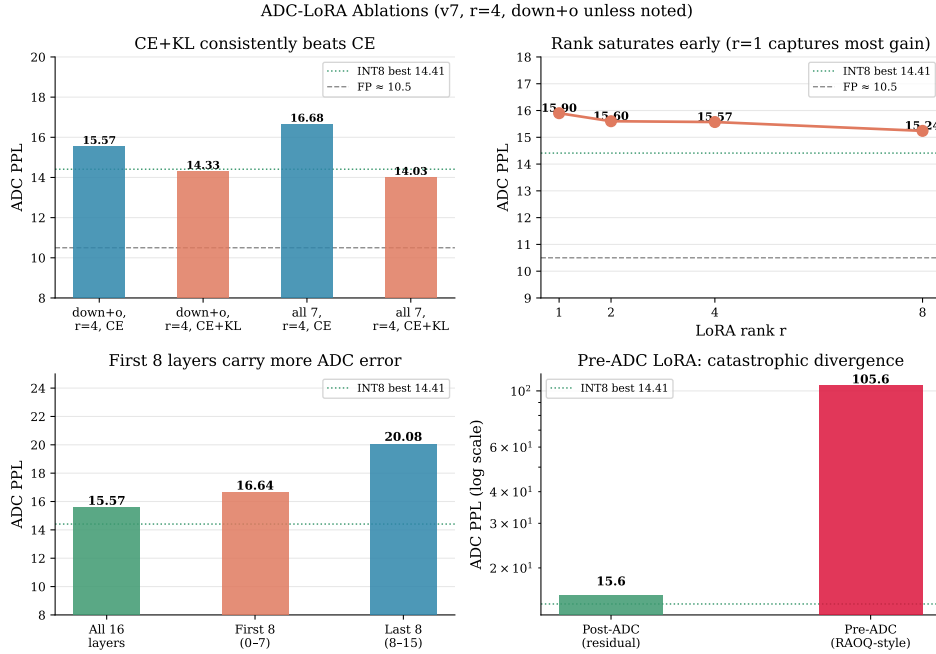


Figure 9: LoRA ablation grid. *Top-left*: CE+KL beats CE across both target sets. *Top-right*: rank saturates near $r = 4$. *Bottom-left*: layer coverage — first-8 beats last-8, all-16 is best, confirming that ADC error accumulates across depth. *Bottom-right*: pre-ADC placement diverges, post-ADC is stable from epoch 1.

Table 9: LoRA generalisation (v8). WikiText-2 and C4 perplexity.

Config	Wiki bypass	Wiki ADC	C4 bypass	C4 ADC
PTQ (no LoRA)	18.49	27.26	29.41	46.40
$r = 8$, down+o, CE	18.51	15.47	30.48	29.39
$r = 4$, down+o, CE+KL	19.05	14.27	29.37	23.88
$r = 4$, all 7, CE+KL	18.70	13.96	29.10	23.30

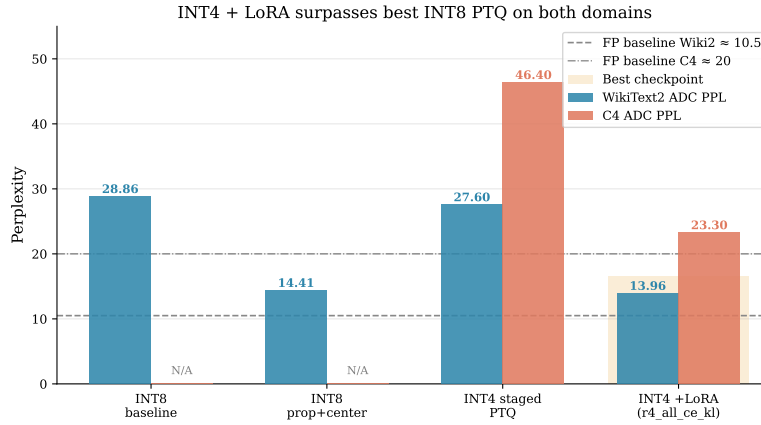


Figure 10: Final results across configurations on WikiText-2 and C4. INT4 + LoRA (all-7, CE+KL, $r = 4$) surpasses the best INT8 PTQ on WikiText-2 (13.96 vs. 14.41) and reduces C4 ADC PPL by 50% relative to the INT4 PTQ baseline (46.40 $\rightarrow$ 23.30).

negative values. A physically important alternative is a unipolar (unsigned) ADC, which accepts only non-negative inputs. Real crossbar hardware is often constrained to non-negative currents, making this the more realistic operating regime. The two settings have the same bit-width $b_a = 8$ and tile size $M = 256$, but they differ in the ADC range and in how the signed dot product is recovered — which changes δ by more than a factor of two and requires a different calibration objective. We therefore treat the unipolar setting as a self-contained experiment series with its own baselines, calibration, and ablation structure.

5.4.1 Unsigned Shift-Subtract ADC Algorithm

Because the crossbar accepts only $[0, 2^b - 1]$ codes, the standard signed INT4 codes $\hat{x} \in [-8, 7]$ and $\hat{w} \in [-8, 7]$ must be shifted to unsigned before the MVM. The algorithm proceeds in five steps.

Step 1 — Shift to unsigned. Each signed code is offset by the zero-point $z = 2^{b_x-1} = 8$:

$$\hat{x}^u = \hat{x} + 8 \in [0, 15], \quad \hat{w}^u = \hat{w} + 8 \in [0, 15].$$

Step 2 — Hardware MVM. The crossbar computes the non-negative dot product over one tile of width $M = 256$:

$$y^u = \langle \hat{x}^u, \hat{w}^u \rangle \in [0, M(2^{b_x} - 1)(2^{b_w} - 1)] = [0, 57600].$$

Because $y^u \geq 0$ always, a single unsigned MVM suffices with no four-quadrant splitting.

Step 3 — ADC floor. The unipolar ADC step size is:

$$\delta_u = \frac{M(2^{b_x} - 1)(2^{b_w} - 1)}{2^{b_a} k} = \frac{256 \cdot 15 \cdot 15}{256 \cdot 16} = 14.0625 \quad (k = 16),$$

and the quantised output is $\hat{y}^u = \lfloor y^u / \delta_u \rfloor \cdot \delta_u$. Note that $\delta_u \approx 14.06$ is more than twice the bipolar $\delta \approx 6.12$ at the same k , reflecting the fact that the same 2^{b_a} bins now cover a range that is roughly $2\times$ wider in absolute terms.

Step 4 — Digital correction (exact). Expanding the unsigned product gives $\hat{x}^u \hat{w}^u = \hat{x} \hat{w} + 8\hat{w} + 8\hat{x} + 64$, so the signed inner product is recovered exactly by:

$$y_{\text{int}} = \hat{y}^u - 8 \sum_j \hat{w}_j^u - 8 \sum_i \hat{x}_i^u + M \cdot 64,$$

where $\sum_j \hat{w}_j^u$ is precomputed once at weight-load time and $\sum_i \hat{x}_i^u$ is computed once per token. The correction involves no approximation and adds negligible digital overhead.

Step 5 — Dequantisation. $y_{\text{real}} = y_{\text{int}} \cdot s_x \cdot s_w$, where s_x is the per-token activation scale and s_w the per-channel weight scale, exactly as in the bipolar setting.

5.4.2 Per-Layer k Search

Because the coarser δ_u (at global $k = 16$) imposes a heavier resolution penalty than the bipolar setting, we introduce a per-layer k search that selects the ADC parallelism independently for each projection layer.

After FlatQuant calibration with a global k , the pre-ADC values y^u are captured on the calibration set for each layer. The minimum k that keeps the ADC range from saturating is found by:

$$R_\ell = \text{quantile}_{0.999}(|y_\ell^u|), \quad k_\ell^* = \min \left\{ k \in \mathcal{K} : k \geq \frac{M(2^{b_x} - 1)(2^{b_w} - 1)}{2R_\ell} \right\},$$

where $\mathcal{K} = \{4, 8, 16, 32, 64\}$. Layers with a narrow output distribution receive a small k (coarser δ , fewer ADC reads); layers with wide distributions receive a large k (finer δ , more reads). The search is run in-place on the fixed PTQ checkpoint without retraining FlatQuant transforms.

5.4.3 Calibration Objective for Unipolar ADC

FlatQuant transforms calibrated for the bipolar $\delta \approx 6.12$ are mismatched to the unipolar $\delta_u \approx 14.06$: the transforms shape the pre-ADC signal for a finer resolution than the hardware does not provide. In the unipolar series, FlatQuant is therefore retrained with the unipolar δ_u formula as the active ADC step during calibration. All other settings (staged training, $\alpha = 0.5$, diagonal scaling) are carried over from the best bipolar configuration.

5.4.4 Experiment Configurations

The following six configurations are evaluated for Llama-3.2-1B on WikiText-2 and C4:

- `fp` — full-precision FP16 baseline.
- `int4_no_adc` — INT4 FlatQuant with the ADC floor disabled; measures FlatQuant transform quality alone. Trained with the bipolar formula (calibration does not see ADC noise), giving WikiText-2 PPL 12.56.
- `best_ptq` — INT4 FlatQuant retrained with δ_u , global $k = 16$.
- `best_ptq_k` — `best_ptq` with per-layer k search applied in-place (no FlatQuant retraining).
- `best_ptq_k_recal` — `best_ptq_k` followed by retraining FlatQuant with the found per-layer k values baked in; tests whether retrained transforms improve over in-place k search.
- `best_lora` — `best_ptq` plus post-ADC residual LoRA ($r = 4$, all 7 projections, CE+KL loss, $\lambda = 0.5$, temperature 2.0, 5 epochs at 10^{-4}).
- `best_lora_k` — `best_lora` plus per-layer k search; the LoRA adapters are trained on top of the k -searched PTQ checkpoint.

5.4.5 Experiment History and Hardware Setting Evolution

The unipolar series was reached through three prior hardware-setting iterations, each of which exposed a design flaw in the preceding one. We document them here because they clarify why the final configuration is necessary, not arbitrary.

Bipolar ADC (signed accumulation). The original setting uses a signed ADC that reads $y_{\text{int}} \in [-M, +M]$ directly. FlatQuant calibration uses $\delta \approx 6.12$ (bipolar formula). This gives `best_ptq` WikiText-2 PPL 26.78 and `best_lora` PPL 14.01 — the reference for the signed regime.

Four-quadrant unipolar. An earlier attempt at unipolar hardware splits signed codes into positive and negative parts and runs four separate non-negative MVMs (x^+w^+ , x^-w^- , x^+w^- , x^-w^+). This achieves $\delta_{\text{branch}} \approx 3.06$ (twice as fine as bipolar) at the cost of $4\times$ the ADC reads. FlatQuant was trained for the bipolar δ , so the calibration objective was still mismatched; `best_ptq` PPL was 18.08.

Mismatched unsigned evaluation (historical). An intermediate step evaluated the unsigned shift-subtract hardware model using FlatQuant checkpoints trained for the bipolar $\delta \approx 6.12$. The mismatch caused `best_ptq` PPL to explode to 94274 because transforms shaped the pre-ADC signal for a step size six times finer than the actual hardware provides. Per-layer k search partially recovered (PPL 16.78) by selecting small k to bring δ back into range, but this was an incorrect setup and the results were not used.

Unipolar FQ — current. The correct configuration retrains FlatQuant with the unipolar δ_u formula from the start. Table 10 summarises the results. `best_ptq_k_recal` is worse than `best_ptq_k`, indicating that retraining FlatQuant on fixed per-layer k values over-specialises the transforms and hurts generalisation; in-place k search after global- k training is preferable.

Table 10: Unipolar (unsigned) ADC results on Llama-3.2-1B. $\delta_u = 14.0625$ at global $k = 16$.

Config	Description	Wiki PPL	C4 PPL
<code>fp</code>	FP16, no quantisation	8.68	13.13
<code>int4_no_adc</code>	INT4 FQ, ADC disabled	12.56	20.13
<code>best_ptq</code>	INT4 FQ + unipolar ADC, $k=16$	39.00	67.78
<code>best_ptq_k</code>	+ per-layer k search	21.64	33.79
<code>best_ptq_k_recal</code>	+ FQ retrained with per-layer k	22.19	36.94
<code>best_lora</code>	+ post-ADC LoRA (global $k=16$)	17.29	29.26
<code>best_lora_k</code>	+ LoRA & per-layer k	14.56	23.78

6 Discussion

6.1 Interpretation of Results

Purpose. Move from raw results to explanation.

What to include. Discuss why certain modifications work and others do not. Relate the observed behavior back to the theory of ADC-constrained output quantization and the internal statistics of LLaMA layers.

6.2 Limits of PTQ Under ADC Constraints

Purpose. State clearly where the chosen approach succeeds and where it fails.

What to include. This subsection should answer whether reconstruction-based PTQ remains sufficient or whether the problem fundamentally requires some degree of model adaptation in the low-bit regime.

6.3 Practical Implications

Purpose. Explain what the results mean beyond the specific experiments.

What to include. Discuss what your findings imply for IMC deployment, for the design of ADC-aware quantization methods, and for future hardware-software co-design of transformer inference systems.

6.4 Threats to Validity

Purpose. Strengthen the scientific quality of the thesis.

What to include. Mention limitations such as reliance on simulation, restricted model sizes, calibration dataset choice, and the gap between simplified hardware assumptions and a real chip implementation.

7 Conclusion

Purpose. Provide a concise answer to the research questions and summarize the thesis contributions.

What to include. Restate the original problem, summarize the method and experimental findings, state the main conclusion about ADC-aware PTQ for LLaMA-family models, and list a few concrete directions for future work such as better output-aware objectives, mixed precision, or lightweight adaptation.

References

- [1] Yamato Arai and Yuma Ichikawa. “Quantization Error Propagation: Revisiting Layer-Wise Post-Training Quantization”. In: *arXiv preprint arXiv:2504.09629* (2025).
- [2] Saleh Ashkboos et al. “QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs”. In: *arXiv preprint arXiv:2404.00456* (2024).
- [3] Yelysei Bondarenko, Riccardo Del Chiaro, and Markus Nagel. “Low-Rank Quantization-Aware Training for LLMs”. In: *arXiv preprint arXiv:2406.06385* (2024).
- [4] Authors omitted here for brevity. “LQER: Low-Rank Quantization Error Reconstruction for LLMs”. In: *arXiv preprint arXiv:2402.02446* (2024).
- [5] Authors omitted here for brevity. “Reshape and Adapt for Output Quantization (RAOQ)”. In: *OpenReview / conference version*. Use your preferred OpenReview BibTeX entry. 2024.
- [6] Julian Büchel et al. “Analog Foundation Models”. In: *arXiv preprint arXiv:2505.09663* (2025).
- [7] Yuji Chai et al. “INT2.1: Towards Fine-Tunable Quantized Large Language Models with Error Correction through Low-Rank Adaptation”. In: *arXiv preprint arXiv:2306.08162* (2023).
- [8] Mengzhao Chen et al. “EfficientQAT: Efficient Quantization-Aware Training for Large Language Models”. In: *arXiv preprint arXiv:2407.11062* (2024).
- [9] Yuxuan Chen et al. “FlatQuant: Flatness Matters for LLM Quantization”. In: *arXiv preprint arXiv:2410.09426* (2024). URL: <https://arxiv.org/abs/2410.09426>.
- [10] Jungwook Choi et al. “PACT: Parameterized Clipping Activation for Quantized Neural Networks”. In: *arXiv preprint arXiv:1805.06085* (2018).
- [11] Tim Dettmers et al. “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale”. In: *arXiv preprint arXiv:2208.07339* (2022).
- [12] Xin Ding et al. “CBQ: Cross-Block Quantization for Large Language Models”. In: *arXiv preprint arXiv:2312.07950* (2023).
- [13] Abhimanyu Dubey et al. “The Llama 3 Herd of Models”. In: *arXiv preprint arXiv:2407.21783* (2024). URL: <https://arxiv.org/abs/2407.21783>.
- [14] Steven K. Esser et al. “Learned Step Size Quantization”. In: *arXiv preprint arXiv:1902.08153* (2019).
- [15] Elias Frantar et al. “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers”. In: *arXiv preprint arXiv:2210.17323* (2022).
- [16] Amir Gholami et al. “A Survey of Quantization Methods for Efficient Neural Network Inference”. In: *arXiv preprint arXiv:2103.13630* (2021).
- [17] Edward J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2022. URL: <https://arxiv.org/abs/2106.09685>.
- [18] Benoit Jacob et al. “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference”. In: *arXiv preprint arXiv:1712.05877* (2018).
- [19] Manuel Le Gallo et al. “AnalogNAS: A Neural Network Design Framework for Accurate Inference with Analog In-Memory Computing”. In: *arXiv preprint arXiv:2305.10459* (2023).

- [20] Ji Lin et al. “AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration”. In: *arXiv preprint arXiv:2306.00978* (2023).
- [21] Haotian Liu et al. “LLM-QAT: Data-Free Quantization Aware Training for Large Language Models”. In: *arXiv preprint arXiv:2305.17888* (2023).
- [22] Ruikang Liu et al. “FlatQuant: Flatness Matters for LLM Quantization”. In: *arXiv preprint arXiv:2410.09426* (2024).
- [23] Zechun Liu et al. “SpinQuant: LLM Quantization with Learned Rotations”. In: *arXiv preprint arXiv:2405.16406* (2024).
- [24] Stephen Merity et al. “Pointer Sentinel Mixture Models”. In: *International Conference on Learning Representations (ICLR)*. 2017. URL: <https://arxiv.org/abs/1609.07843>.
- [25] Markus Nagel et al. “Up or Down? Adaptive Rounding for Post-Training Quantization”. In: *arXiv preprint arXiv:2004.10568* (2020).
- [26] Reiner Pope et al. “Efficiently Scaling Transformer Inference”. In: *Proceedings of Machine Learning and Systems* 5 (2023).
- [27] Colin Raffel et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67. URL: <https://arxiv.org/abs/1910.10683>.
- [28] Meyer Scetbon and James Hensman. “Low-Rank Correction for Quantized LLMs”. In: *arXiv preprint arXiv:2412.07902* (2024).
- [29] Wenqi Shao et al. “OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models”. In: *arXiv preprint arXiv:2308.13137* (2023).
- [30] Xiuying Wei et al. “Outlier Suppression+: Accurate Quantization of Large Language Models by Equivalent and Optimal Shifting and Scaling”. In: *arXiv preprint arXiv:2304.09145* (2023).
- [31] Xiuying Wei et al. “QDrop: Randomly Dropping Quantization for Extremely Low-bit Post-Training Quantization”. In: *arXiv preprint arXiv:2203.05740* (2022).
- [32] Hao Wu et al. “Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation”. In: *arXiv preprint arXiv:2004.09602* (2020).
- [33] Guangxuan Xiao et al. “SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models”. In: *arXiv preprint arXiv:2211.10438* (2022).
- [34] Patrick Xiao et al. “On the Accuracy of Analog Neural Network Inference Accelerators”. In: *arXiv preprint arXiv:2109.01262* (2021).
- [35] Zhewei Yao et al. “ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers”. In: *arXiv preprint arXiv:2206.01861* (2022).

A Detailed Experimental Configurations

Suggested contents. Put complete hyperparameter tables here: number of calibration samples, batch size, learning rates, transform settings, clipping settings, hardware parameters, and final selected configurations for each run.

B Additional Figures and Tables

Suggested contents. Add extended ablation tables, per-layer plots, extra sensitivity analyses, and any visualizations that support the main text but are too large to place in the core chapters.

C Code Organization and Reproducibility Notes

Suggested contents. Briefly describe the repository structure, entry points for training and evaluation, external dependencies, and the exact steps required to reproduce the reported results.

D Key Code Listings

This appendix collects the four code components that implement the method described in Sections 4. All listings are simplified for readability — error handling, logging, and BitAug / bypass switches are omitted; the full code is available in the repository at `ADC/llama/core/`.

D.1 ADC Forward Pass

Listing 1 shows the per-tile ADC forward pass from `QATLinearADC.forward` (file `adc_layers.py`). It implements equations (16) and the dead-zone / clipping behaviour: per-token activation scale, per-channel weight scale, integer MVM in code domain, ADC floor + clamp, and dequantisation. The step size δ is fixed at construction from (M, q_x, q_w, b_a, k) .

```
1 def forward(self, x):
2     # Per-token activation scale s_x (amax over feature dim)
3     s_x = x.abs().amax(dim=-1, keepdim=True).clamp(min=1e-6) / act_levels
4     code_x = round_ste(x / s_x).clamp(qmin_x, qmax_x)
5
6     # Per-channel weight scale s_w
7     code_w = round_ste(safe_divide(self.weight, s_w_vec.view(-1, 1)))
8     code_w = torch.clamp(code_w, qmin_w, qmax_w)
9
10    # Integer MVM in code domain
11    y_int = F.linear(code_x, code_w, bias=None)
12
13    # ADC: floor + clamp. delta = (2 * M * q_x * q_w) / (2^b_a * k)
14    if self.bypass_adc:
15        adc_output = y_int
16    else:
17        y_adc_codes = floor_ste(y_int / self.delta)
18        y_adc_codes = torch.clamp(y_adc_codes, self.na, self.pa)
19        adc_output = y_adc_codes * self.delta
20
21    # Dequantise: s_x [B,1] * s_w_vec [out] -> [B, out]
```

```

22     y_real = adc_output * s_x * s_w_vec
23     if self.bias is not None:
24         y_real = y_real + self.bias
25     return y_real

```

Listing 1: Per-tile ADC forward pass (QATLinearADC.forward).

TiledLinearADC wraps several QATLinearADC tiles side-by-side along the input dimension whenever the projection width exceeds the crossbar size M (the `mvm_limit`). Each tile has its own δ derived from its local in-features.

D.2 Kronecker Transform with Diagonal

Listing 2 shows the trainable Kronecker transform $T = T_1 \otimes T_2$ together with the optional per-channel diagonal D (file `flat_quant.py`, class `KroneckerTransform`). Each Kronecker factor is parameterised as $U \text{diag}(d) V^\top$ with U, V kept orthogonal via PyTorch’s `matrix_exp` parametrisation, which preserves orthogonality under unconstrained gradient updates. The `add_diag` branch implements $\tilde{T} = T \cdot D$ from Section 4.

```

1  def forward(self, x, inv_t=False):
2      # Optional per-channel diagonal D (Section 4.5)
3      if self.add_diag and self.use_diag:
4          d = self.diag_scale.abs().clamp(min=1e-8)
5          x = x / d if inv_t else x * d
6
7      # Left and right Kronecker factors  $T_i = U_i * \text{diag}(d_i) * V_i^\top$ 
8      dl = self.diag_left.abs().clamp(min=1e-6)
9      dr = self.diag_right.abs().clamp(min=1e-6)
10     if inv_t:
11         dl, dr = 1.0 / dl, 1.0 / dr
12     mat_l = self.u_left.weight @ torch.diag(dl) @ self.v_left.weight.T
13     mat_r = self.u_right.weight @ torch.diag(dr) @ self.v_right.weight.T
14
15     #  $x_{\text{reshaped}} @ (T_1(x) T_2) = T_1^\top \text{reshape } T_2 \text{ reshape}$ 
16     return _kronecker_matmul(x, mat_l, mat_r)

```

Listing 2: Kronecker transform with optional diagonal scaling (KroneckerTransform.forward).

At deployment, `to_eval_mode` pre-computes and caches the full forward, inverse, and transpose matrices and discards the parametrised U, V tensors. The transforms are then folded into the linear weights via `reparameterize`, leaving zero inference overhead.

D.3 Block-Level Loss with α -Mixing

Listing 3 shows the block-level reconstruction loss with the dual-forward α -mixed objective from Section 4. The same block runs twice — once on the clean FP teacher input x^* and once on the ADC-quantised output $\hat{x}$ from the previous block — and the two losses are combined linearly.

```

1  # y_star: teacher block output on clean FP input (no grad)
2  # x_clean: clean FP activations
3  # x_adc: ADC-quantised activations from previous block (propagation)
4  y_fp = block(x_clean) # clean forward through quantised
    block

```

```

5 y_adc = block(x_adc)                                # propagated forward (sees real ADC
   noise)
6
7 loss_fp = F.mse_loss(y_fp, y_star)
8 loss_adc = F.mse_loss(y_adc, y_star)
9
10 # Eq. (4.X): L = (1 - alpha) * L_FP + alpha * L_ADC
11 loss = (1.0 - alpha) * loss_fp + alpha * loss_adc

```

Listing 3: Block-level α -mixed reconstruction loss.

Staged training (Section 4.1.6) is implemented by simply varying which parameters require gradients and which α is used per stage:

```

1 # Stage A: 30 epochs, MLP factors only, alpha = 0
2 for p in mlp_kron_and_diag_params: p.requires_grad_(True)
3 for p in attn_kron_and_diag_params: p.requires_grad_(False)
4 train(epochs=30, lr=eta, alpha=0.0)
5
6 # Stage B: 10 epochs, attn unfrozen, alpha = 0.5
7 for p in attn_kron_and_diag_params: p.requires_grad_(True)
8 train(epochs=10, lr=0.1 * eta, alpha=0.5)

```

Listing 4: Staged training schedule (Stage A $\rightarrow$ Stage B).

D.4 Post-ADC Residual LoRA

Listing 5 shows the post-ADC residual LoRA wrapper (file `adc_lora.py`, class `ResidualLoRATiledLinearADC`). The frozen `TiledLinearADC` produces the quantised path; a parallel rank- r branch reads the FP input and adds its float32 output to the ADC result. `lora_B` is initialised to zero so the wrapped layer initially matches the frozen PTQ checkpoint exactly.

```

1 class ResidualLoRATiledLinearADC(nn.Module):
2     def __init__(self, tiled_layer, r=4, alpha=8.0):
3         super().__init__()
4         self.tiled_layer = tiled_layer                # frozen ADC path
5         self.scaling = alpha / r
6         self.lora_A = nn.Linear(tiled_layer.in_features_total, r,
7                                 bias=False, dtype=torch.float32)
8         self.lora_B = nn.Linear(r, tiled_layer.out_features,
9                                 bias=False, dtype=torch.float32)
10        nn.init.kaiming_uniform_(self.lora_A.weight, a=math.sqrt(5))
11        nn.init.zeros_(self.lora_B.weight)            # zero start
12        for p in self.tiled_layer.parameters():
13            p.requires_grad_(False)                   # freeze base path
14
15        def forward(self, x):
16            y_base = self.tiled_layer(x)               # frozen ADC path
17            y_lora = self.lora_B(self.lora_A(x.float())) * self.scaling
18            return y_base + y_lora.to(y_base.dtype)

```

Listing 5: Post-ADC residual LoRA wrapper (`ResidualLoRATiledLinearADC`).

Pre-ADC placement is implemented by the sister class `PreADCLoRATiledLinearADC`, which adds the rank- r update to the weights of each tile *before* integer quantisation and ADC. As shown in Section 5, this placement diverges catastrophically (ADC PPL ~ 105) and is included only as an ablation.